Project - High Level Design
on
Multi-Agent Education System

Course Name: Agentic AI

Institution Name: Medicaps University – Datagami Skill Based Course

Student Name(s) & Enrolment Number(s):

Sr no	Student Name	Enrolment Number
1	Leeni Patidar	EN22CS301554
2	Mahak Agrawal	EN22CS301563
3	Kunal Singh Chouhan	EN22CS301540
4	Krishna Patidar	EN22CS301523

Group Name: 04D5

Project Number: AAI-04

Industry Mentor Name:

University Mentor Name:: Prof. Maya Yadav

Academic Year: 2025-2026

1.Introduction

The **Multi-Agent AI Education System (NoteMind AI)** is an intelligent AI-powered learning assistant designed to automatically generate structured and high-quality educational content from user-provided topics. The primary objective of the system is to enhance student productivity and learning efficiency by transforming raw subject inputs into organized study materials in multiple formats.

The platform enables users to generate detailed explanatory notes, concise summaries, bullet-point study material, and quiz questions for self-assessment. Additionally, users can download generated notes in DOCX format and access previously generated content through a personalized history feature.

Unlike traditional AI chatbots that produce unstructured and context-limited responses, this system leverages a multi-agent architecture integrated with Retrieval-Augmented Generation (RAG) and a locally deployed Large Language Model (LLM). This combination ensures structured, context-aware, and semantically grounded outputs suitable for academic use.

The system integrates multiple modern AI components, including agent orchestration, semantic retrieval, and offline LLM execution, making it both scalable and privacy-conscious. By running the language model locally, the platform ensures enhanced data privacy while reducing dependency on external APIs

1.1Scope of the Project

Included in Scope

The following components are implemented as part of the system:

• Multi-agent architecture for modular AI intelligence
• Retrieval-Augmented Generation (RAG) pipeline for contextual learning
• Web search integration for real-time knowledge retrieval
• Local LLM execution using Ollama
• Secure authentication (Google OAuth and local login)
• User history storage and retrieval
• DOCX export functionality for offline usability

Out of Scope

The following features are not included in the current implementation:

• Training or fine-tuning custom Large Language Models
• Native mobile application development
• Real-time collaborative editing among users
• Fully distributed cloud deployment architecture

These exclusions help define the boundaries of the project and maintain focus on core AI system design.

1.2Intended Audience

This documentation is designed for both technical and academic stakeholders.

Primary Audience

• Software developers building AI-powered applications
• AI/ML engineers exploring multi-agent systems
• System architects studying intelligent software architectures

Secondary Audience

• Students using the learning assistant
• Academic mentors evaluating the system
• Project reviewers and technical evaluators

Prerequisites

To fully understand the concepts presented in this system, readers should possess:

• Basic understanding of Python programming
• Familiarity with Large Language Models (LLMs)
• Awareness of Retrieval-Augmented Generation (RAG) concepts
• General understanding of AI-powered applications

1.3System Overview

The Multi-Agent AI Education System follows an agent-driven orchestration model in which multiple specialized AI agents collaborate to transform a simple topic into structured study material.

Rather than relying on a single monolithic model, the system distributes responsibilities across multiple intelligent agents, each handling a dedicated function such as research, retrieval, and structured writing. This modular design enhances scalability, maintainability, and output quality.

By combining multi-agent orchestration with semantic retrieval and local LLM execution, the platform delivers accurate, context-aware educational content while maintaining operational efficiency and privacy.

Core Workflow

The workflow begins with user authentication, where the system verifies identity and creates a personalized session.

i. The authenticated user enters a topic and selects the desired output format through the web interface.

ii.The multi-agent orchestration layer is activated to coordinate specialized AI agents.

iii.The Research Agent gathers relevant information from external sources such as web search results and internal knowledge repositories.
iv.The Retrieval Agent implements the Retrieval-Augmented Generation (RAG) pipeline.
v.The input topic is converted into semantic embeddings using an embedding model.
A similarity search is performed on the vector database to retrieve contextually relevant information.
vi.The retrieved contextual data is aggregated and passed to the Writer Agent.
vii.The Writer Agent generates structured educational content using a local Large Language Model.
viii.The generated output is formatted based on the user-selected format for clarity and readability.
ix.The final output is displayed on the user interface.

x.The generated content is stored in the user's personalized history for future access.
An optional export feature allows users to download the generated notes as a structured document.
xi.The workflow ensures a seamless transition from topic input to structured knowledge output while maintaining modularity and scalability.

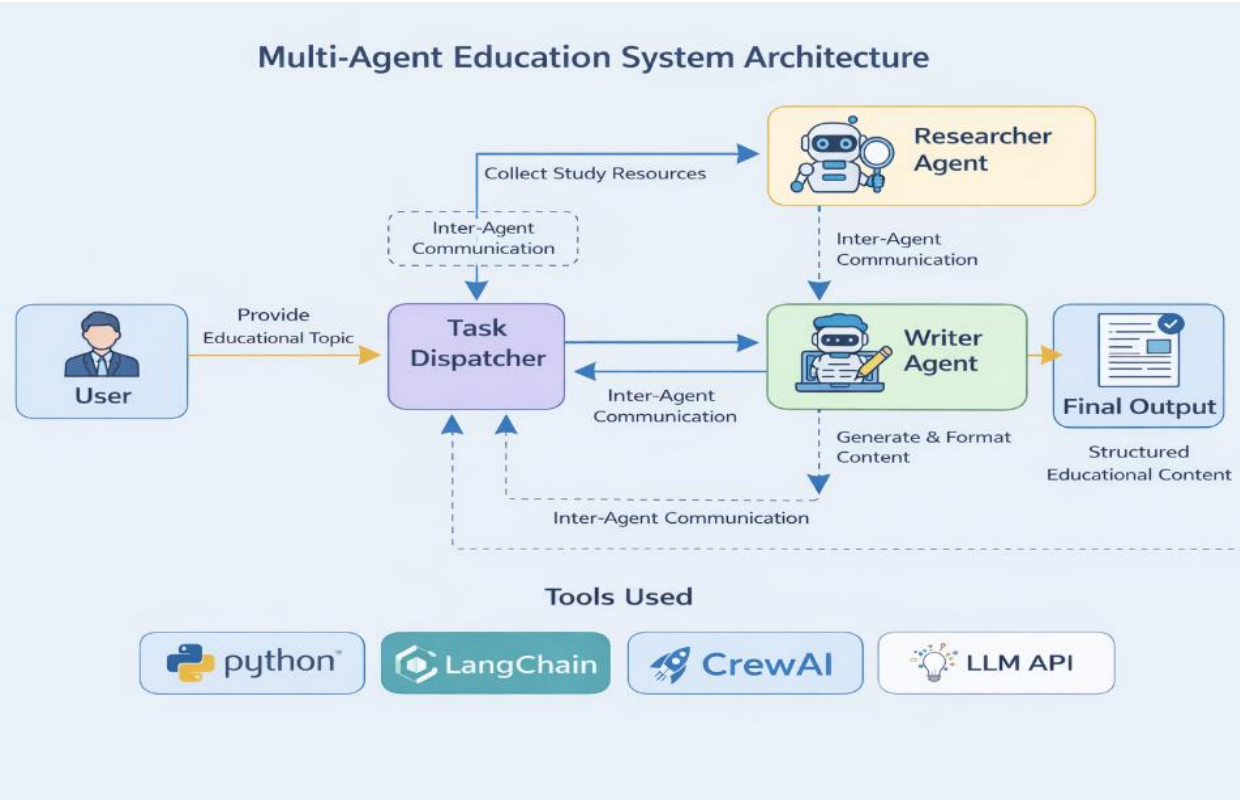
2.System Design

Application Type:

Multi-Agent AI Education Assistant
Users:
The primary users of the system are students who want to generate structured study materials quickly using AI.

Inputs:
The system takes the following inputs from the user: topic name, note type (detailed notes, short notes, or bullet points), question type (MCQs, long questions, or short questions), and quiz generation requests.

Outputs:
The system generates structured study content such as AI-generated notes, summaries, quizzes, and downloadable DOCX files. The generated content is also stored in the user's history for future reference.



2.1Application Design:

The application is designed as a modular multi-layer AI system consisting of a frontend interface, backend processing layer, multi-agent orchestration layer, and knowledge retrieval components. The frontend is built using Streamlit and provides an interactive user interface where users can log in, enter a topic, select the type of output, and view generated notes. The backend is implemented in Python and integrates multiple AI modules. A multi-agent orchestration framework coordinates different specialized agents responsible for research, knowledge retrieval, and content generation. The system uses a local Large Language Model through Ollama, enabling offline operation and eliminating reliance on paid APIs. A Retrieval-Augmented Generation (RAG) pipeline enhances accuracy by converting text into embeddings using a transformer-based embedding model and storing them in a FAISS vector database for semantic search. When required, a web search module fetches real-time information to enrich generated content. The system also includes secure authentication using Google OAuth and local login, user history persistence, and a document export layer that allows generated notes to be downloaded in DOCX format. This layered and modular architecture ensures scalability, cost efficiency, and high-quality AI-generated educational outputs.

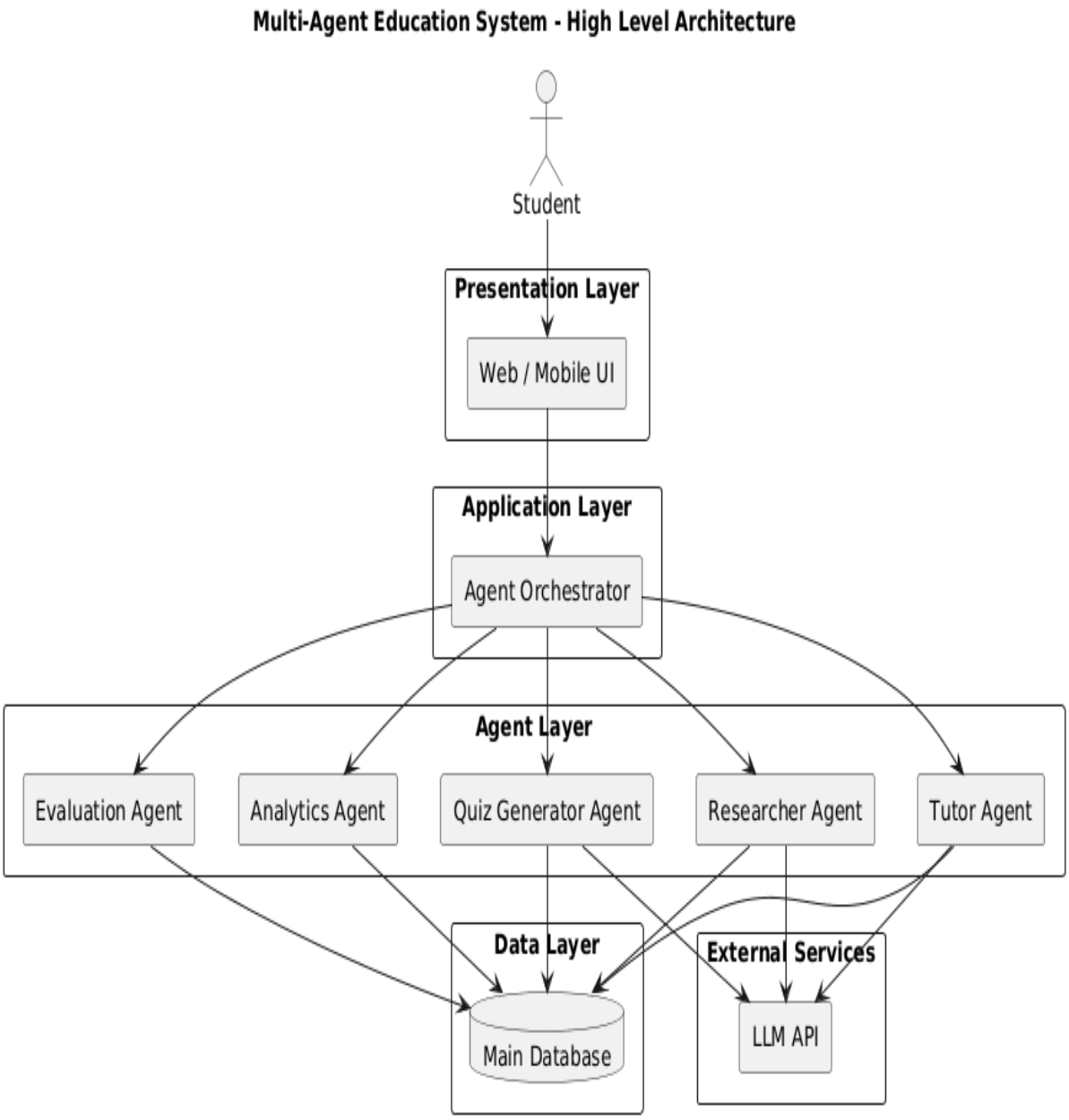


Fig:2.1-Application Design

2.2Process Flow Explanation:

The user logs into the application using Google OAuth or local authentication and enters a topic along with the desired output format. The request is passed to the multi-agent orchestration layer, where specialized agents handle different tasks. The research component gathers relevant information from external sources, while the knowledge retrieval component searches the vector database using semantic similarity. The retrieved context is then provided to the language model, which generates structured notes, summaries, or quizzes based on the

selected format. The generated output is displayed in the user interface, stored in the user's history, and can optionally be exported as a DOCX file for offline use.

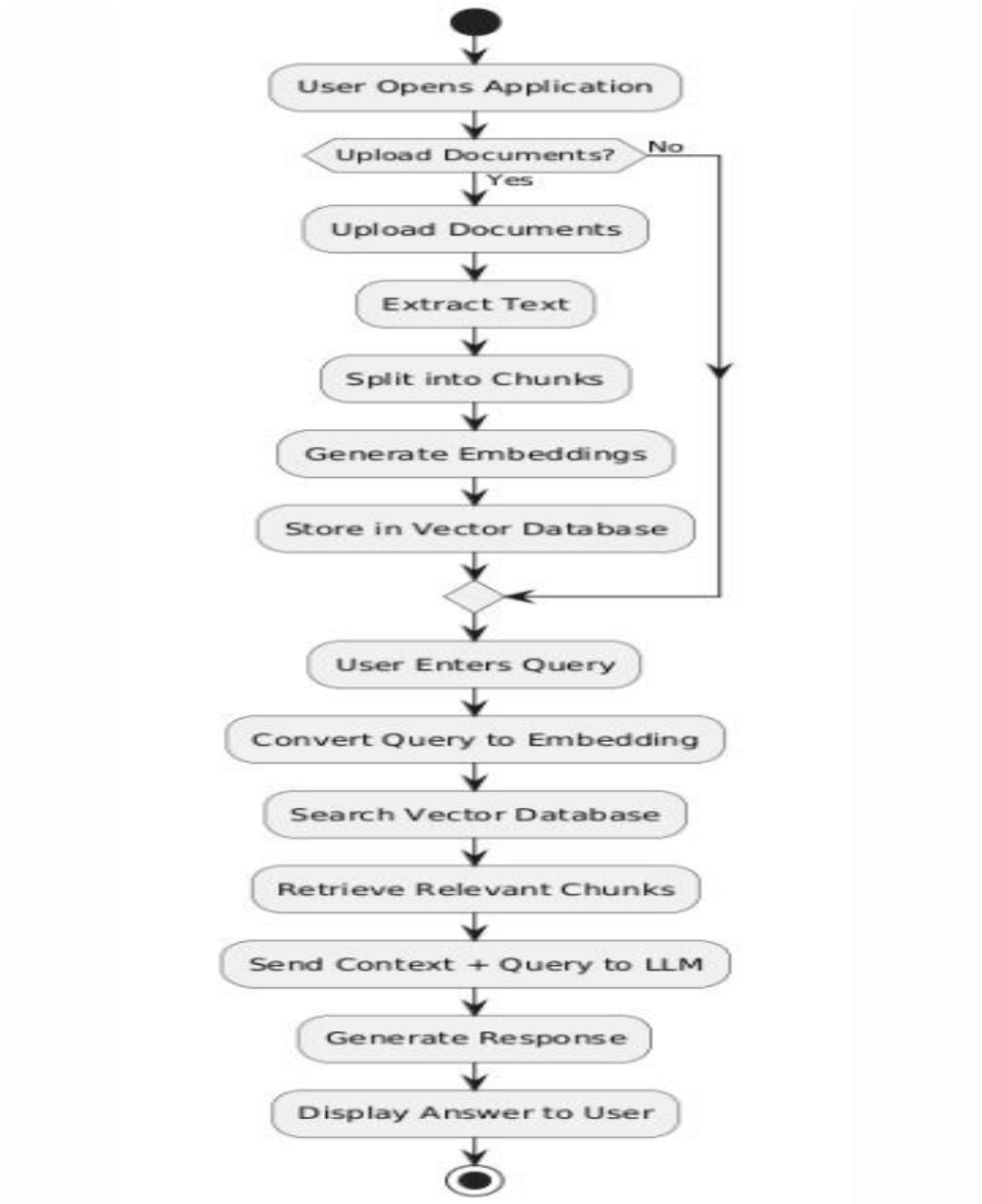
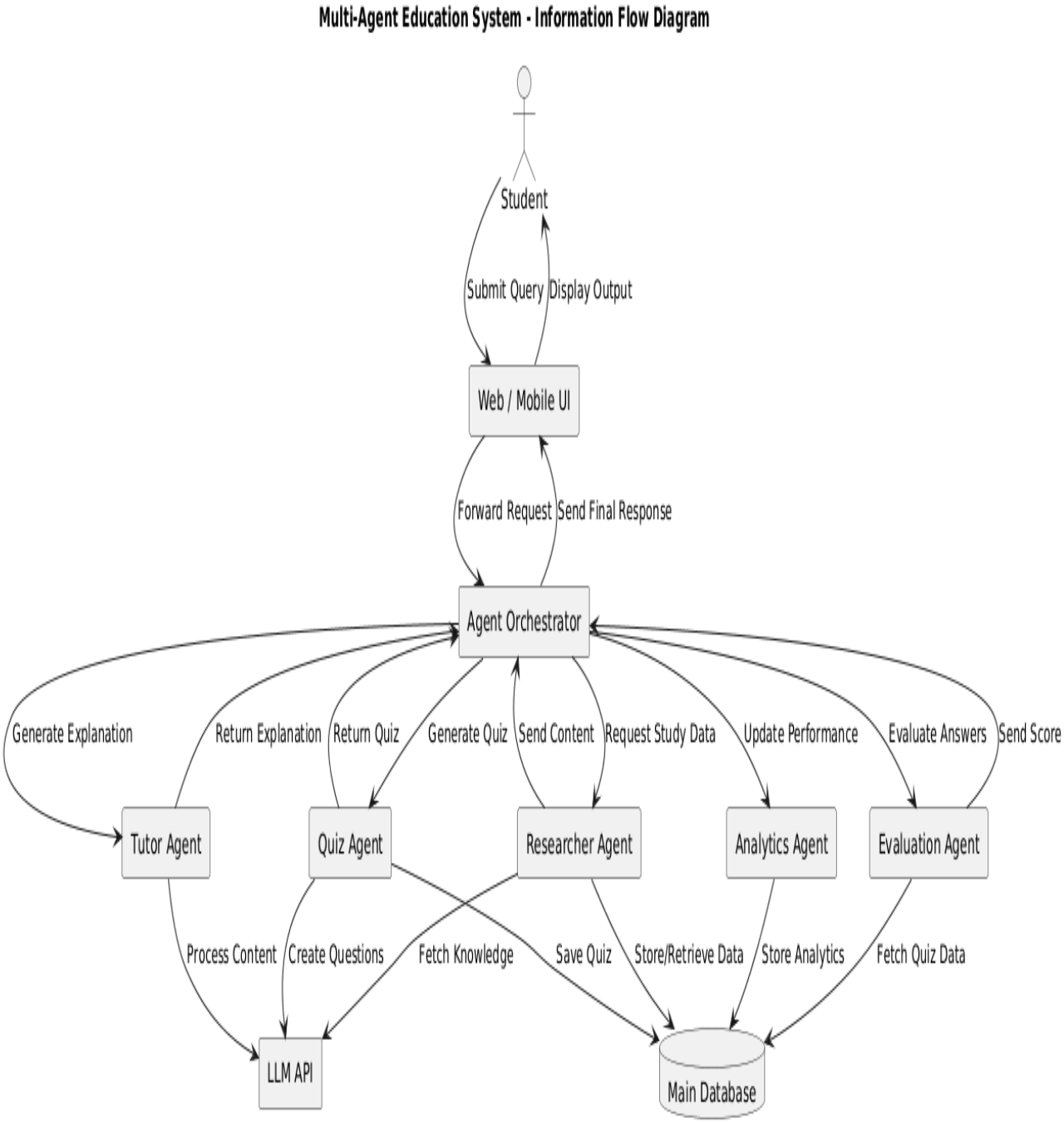


Fig:2.2-Process Flow

2.3Information Flow:

The system begins by accepting user input and validating authentication. The multi-agent pipeline is then triggered, which performs web search and semantic retrieval. The retrieved information is processed by the language model to generate structured content. The output is displayed in the frontend, stored in persistent history storage, and made available for export.



2.4Key Design Considerations:

The system is designed with a modular multi-agent architecture to improve scalability and maintainability. A local LLM is used to reduce operational cost and ensure offline capability. Retrieval-Augmented Generation improves factual accuracy and contextual relevance. Secure

authentication mechanisms protect user data. FAISS-based vector search ensures fast semantic retrieval. The interface is designed to be clean and user-friendly, while persistent storage ensures history tracking and usability.

2.5API Catalogue:

The system follows a pseudo-API architecture for modularity. The generate endpoint handles AI content generation requests. The login endpoint manages user authentication. The history endpoint retrieves previously generated notes for a user. The export endpoint allows generated notes to be downloaded as a DOCX document.

3.Data Design

The data design of the system focuses on efficient handling of user inputs, semantic processing, and structured storage of generated outputs. The system follows a hybrid data model that includes user metadata, generated educational content, and vector-based semantic representations. The primary goal of the data design is to ensure fast retrieval, personalization, and accurate AI-generated responses while maintaining scalability and modularity.

3.1Data Model:

The core data model begins with the user entity, which represents authenticated individuals accessing the platform through Google OAuth or local login. Each user interacts with the system by providing a topic, which acts as the primary input for content generation. Once a topic is submitted, it passes through the multi-agent processing pipeline where specialized agents perform research, retrieval, and structured content generation. The final output is generated in the form of structured notes, summaries, or quizzes. This generated content is then stored in the user's history database, allowing retrieval of previously generated topics and enabling personalization. The history component ensures continuity of learning and allows users to revisit or export their past generated materials.

3.2Vector Data Flow (Semantic Processing Pipeline):

In addition to the traditional data model, the system uses a semantic vector pipeline to improve contextual understanding and retrieval accuracy. When a topic is entered, it is converted into a dense vector representation using a transformer-based embedding model. These embeddings are used to perform similarity searches within a FAISS vector

database that stores previously indexed knowledge representations. The semantic search retrieves the most relevant contextual information related to the user's query. This retrieved context is then passed to the language model as additional grounding data. The Large Language Model processes both the user query and retrieved context to generate coherent and structured final output. This vector-based data flow ensures improved relevance, reduced hallucination, and higher semantic alignment in generated educational content.

Overall, the combination of structured user data storage and vector-based semantic retrieval enables the system to balance personalization, speed, and accuracy. The hybrid data design ensures that the platform remains scalable while delivering context-aware AI-generated learning materials.

4. Interfaces

The system integrates multiple interfaces to ensure smooth communication between components. The frontend interface is developed using Streamlit, which provides an interactive web-based environment for users to log in, enter topics, select note types, and view generated outputs. The application communicates with the Ollama Local Server to execute the Large Language Model for content generation. A FAISS vector store interface is used to perform semantic similarity searches for context retrieval within the RAG pipeline. The system also connects to the Google OAuth service to enable secure user authentication. Additionally, the DuckDuckGo Search API is integrated to fetch real-time information when required. The export interface allows users to download generated notes in DOCX format, and a history interface enables retrieval of previously generated content.

5.State and Session Management

The system uses Streamlit session state to manage runtime interactions and maintain user-specific data during active sessions. Once a user logs in through Google OAuth or local authentication, the login credentials and session status are securely stored in the session state. User inputs, such as selected topics and generated outputs, are temporarily maintained throughout the session to enable smooth interaction and re-rendering of results. Each user's generated content is stored separately in persistent history storage to ensure personalization and data isolation. The FAISS vector index is loaded at runtime to support efficient semantic retrieval. For security purposes, session data is cleared upon logout to prevent unauthorized access. This structured session management approach ensures data consistency, security, and a seamless user experience.

6. Caching

The system incorporates lightweight caching mechanisms to enhance performance and reduce redundant computations during repeated interactions. The FAISS vector index is loaded into memory at runtime to enable fast semantic similarity searches without repeated disk access.

Retrieved contextual chunks are reused within the same generation cycle to avoid unnecessary repeated retrieval operations. Optional caching strategies can also be applied to repeated user queries, improving response time and system efficiency. These caching techniques reduce latency, optimize resource usage, and ensure a smoother user experience.

Key Caching Points:
• FAISS index kept in memory for faster retrieval
• Retrieved chunks reused during generation process
• Optional caching for repeated queries
• Reduced redundant embedding computations
• Faster response time for similar topics
• Improved overall system performance

7.Non-Functional Requirements

Non-functional requirements define the quality attributes of the system, including security, performance, reliability, and maintainability. These requirements ensure that the system not only functions correctly but also provides a secure, fast, and scalable user experience. The platform is designed with a focus on data privacy, efficient execution, and modular architecture to support future enhancements.

7.1Security:
The system incorporates multiple layers of security to protect user data and ensure safe interactions. Authentication is handled through Google OAuth integration and local login support, ensuring that only authorized users can access the system. Passwords for local authentication are securely hashed using bcrypt to prevent unauthorized access. Additionally, the use of a local Large Language Model ensures that sensitive user queries are not transmitted to external APIs, improving privacy and data control.

• Google OAuth authentication for secure login
• Encrypted password storage using bcrypt
• Local LLM deployment (no external data sharing)
• Session-based access control
• User-specific history isolation

7.2Performance:
The system is optimized for fast and efficient execution using lightweight yet powerful AI components. FAISS is used as the vector database to enable high-speed semantic similarity searches. The use of a local Ollama runtime reduces network latency and ensures faster content generation. A modular multi-agent architecture further improves performance by structuring processing into specialized tasks, enabling efficient orchestration and scalability.

• FAISS for fast vector similarity search
• Local Ollama runtime for low latency

• Modular agents for structured processing
• In-memory vector indexing
• Lightweight Streamlit frontend

8.References

The system is built using modern open-source frameworks and research-backed methodologies. The following references provide technical guidance and foundational knowledge for the technologies and concepts used in the project.

• Streamlit Documentation
• CrewAI Documentation
• Ollama Documentation
• FAISS Documentation